Aeson

Data.Aeson: https://hackage.haskell.org/package/aeson-2.1.1.0

```
-- Data.Aeson
-- Data.ByteString.Lazy
decode :: FromJSON a => ByteString -> Maybe a

decode' :: FromJSON a => ByteString -> Maybe a
                 -- strict convert i.e. ByteString -> a
immediately
eitherDecode :: FromJSON a => ByteString -> Either
String a
eitherDecode' :: FromJSON a => ByteString -> Either
String a

-- Data.ByteString (strict)
decodeStrict :: FromJSON a => ByteString -> Maybe a

decodeStrict' :: FromJSON a => ByteString -> Maybe a
                   -- strict convert i.e. ByteString ->
a immediately
eitherDecodeStrict :: FromJSON a => ByteString -> Either
String a
eitherDecodeStrict' :: FromJSON a => ByteString ->
Either String a

decodeFileStrict :: FromJSON a => FilePath -> IO (Maybe
a)
decodeFileStrict' :: FromJSON a => FilePath -> IO (Maybe
a)
eitherDecodeFileStrict :: FromJSON a => FilePath -> IO
(Either String a)
eitherDecodeFileStrict' :: FromJSON a => FilePath -> IO
(Either String a)

encode :: ToJSON a => a -> ByteString
encodeFile :: ToJSON a => FilePath -> a -> IO ()
```

Types

```
-- Data.Aeson
data Value =
 Object !Object
 | Array !Array
 | String !Text
 | Number !Scientific
 | Bool !Bool
 | Null
instance ToJSON Value, FromJSON Value, IsString
Value, ...
```

```
type Array = Vector Value
 -- Data.Vector
type Object = KeyMap Value
  -- Data.Aeson.KeyMap

type Pair = (Key, Value)
object :: [Pair] -> Value
```

Encoding

```
-- Data.Aeson
class ToJSON a where
 toJSON :: a -> Value
 toEncoding :: a -> Encoding
                   -- Encoding ~ binary builder
Data.Binary.Builder https://hackage.haskell.org/package/
binary
 toJSONList :: [a] -> Value
 toEncodingList :: [a] -> Encoding

class KeyValue kv where
 (.=) :: ToJSON a => Key -> a -> kv
instance KeyValue Series
instance value ~ Value => KeyValue (KeyMap value)
instance (key ~ Key, value ~ Value) => KeyValue (key,
value)


-- Series
data Series
pairs :: Series -> Encoding
foldable :: (Foldable t, ToJSON a) => t a -> Encoding
```

Decoding

```
-- Data.Aeson
class FromJSON a where
  parseJSON :: Value -> Parser a
                    -- Data.Aeson.Types.Parser
  parseJSONList :: Value -> Parser a

data Result = Error String | Success a
fromJSON :: FromJSON a => Value -> Result a

type JSONPath = [JSONPathElement]
data JSONPathElement = Key Key | Index !Int

withObject :: String -> (Object -> Parser a) -> Value ->
Parser a
withText :: String -> (Text -> Parser a) -> Value ->
Parser a
withArray :: String -> (Array -> Parser a) -> Value ->
```

```
Parser a
withScientific :: String -> (Scientific -> Parser a) ->
Value -> Parser a
withBool :: String -> (Bool -> Parser a) -> Value ->
Parser a
withEmbeddedJSON :: String -> (Value -> Parser a) ->
Value -> Parser a

(<?>) :: Parser a -> JSONPathElement -> Parser a
(.:) :: FromJSON a => Object -> Key -> Parser a
(.:?) :: FromJSON a => Object -> Key -> Maybe (Parser a)
(.:!) :: FromJSON a => Object -> Key -> Maybe (Parser a)
(.!=) :: Parser (Maybe a) -> a -> Parser a

json :: Parser Value
json' :: Parser Value
...
```

Defaults + options

```
-- Data.Aeson
genericToJSON :: (Generic a, GToJSON' Value Zero (Rep
a)) => Options -> a -> Value
genericLiftToJSON :: (Generic1 f, GToJSON' Value One
(Rep1 f)) => Options -> (a -> Value) -> ([a] -> Value)
-> f a -> Value
genericToEncoding :: (Generic a, GToJSON' Encoding Zero
(Rep a)) => Options -> a -> Encoding
genericLiftToEncoding :: (Generic1 f, GToJSON' Encoding
One (Rep1 f)) => Options -> (a -> Encoding) -> ([a] ->
Encoding) -> f a -> Encoding
genericParseJSON :: (Generic a, GFromJSON Zero (Rep a))
=> Options -> Value -> Parser a
genericLiftParseJSON :: (Generic1 f, GFromJSON One (Rep1
f)) => Options -> (Value -> Parser a) -> (Value ->
Parser [a]) -> Value -> Parser (f a)

-- conversion options
data Options
defaultOptions ::
Options
fieldLabelModifier :: Options -> String ->
String            -- default: id
constructorModifier :: Options -> String ->
String           -- default: id
allNullaryToStringTag :: Options ->
Bool                     -- default: True
omitNothingFields :: Options ->
Bool                         -- default: False
sumEncoding :: Options ->
SumEncoding
unwrapUnwaryRecords :: Options ->
Bool                       -- default False
tagSingleConstructors :: Options ->
```

```
Bool                      -- default False
rejectUnknownFields :: Options ->
Bool                        -- default False
```

Data.Aeson.Key

```
-- Data.Aeson.Key
data Key
instance
  Monoid Key, IsString Key,
  ToJSON Key, FromJSON Key,
  ToJSONKey Key, FromJSONKey Key, IsString Key, ...

fromString :: String -> Key
fromText :: Text -> Key
toString :: Key -> String
toText :: Key -> Text
```

Data.Aeson.KeyMap

```
-- Data.Aeson.KeyMap
data KeyMap v
empty :: KeyMap v

member :: Key -> KeyMap v -> Bool
lookup :: Key -> KeyMap v -> Maybe v
(!?) :: KeyMap v -> Key -> Maybe v
null :: KeyMap v -> Bool
size :: KeyMap v -> Int

singleton :: Key -> v -> KeyMap v
insert :: Key -> v -> KeyMap v -> KeyMap v
...
```

Data.Aeson.Encoding

```
-- Data.Aeson.Encoding
type Encoding = Encoding' Value
data Encoding' tag

encodingToLazyByteString :: Encoding' a -> ByteString
                      -- Data.ByteString.Lazy
fromEncoding :: Encoding' tag -> Builder
                       -- Data.Binary.Builder
unsafeToEncoding :: Builder -> Encoding' tag



data Series
                        -- encoding of JSON object
pairs :: Series -> Encoding
pair :: Key -> Encoding -> Series
                      -- object key-value
```

```
pairStr :: String -> Encoding -> Series
pair' :: Encoding' Key -> Encoding -> Series
emptyObject_ :: Encoding

list :: (a -> Encoding) -> [a] -> Encoding
emptyArray_ :: Encoding

text :: Text -> Encoding
string :: String -> Encoding
bool :: Bool -> Encoding
int :: Int -> Encoding
... etc
```

Data.Aeson.Types

```
-- Parser
data Parser a
instance MonadFail Parser, MonadFix Parser, Alternative
Parser, MonadPlus Parser
  Monoid (Parser a)

data Result a = Error String | Success a

...
```

Encoding/decoding
Ref: https://hackage.haskell.org/package/aeson-2.1.1.0/docs/Data-Aeson.html#g:14

**Encoding/decoding**

encoding value ==> JSON
implemented with **Encoding type (Data.Aeson.Encoding)**
Uses binary builder **Data.Binary.Builder** https://hackage.haskell.org/package/binary
Converts
a -- Encoding --> ByteString

decoding JSON ==> value
implemented with **Parser type (Data.Aeson.Parser)**
Uses **Data.Attoparsec**: https://hackage.haskell.org/package/attoparsec
Converts
ByteString -- Parser Value --> Value --> a

3 ways to encode/decode:
Method 1) default via **anyclass derive**: suffices for most cases
e.g.

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE DeriveAnyClass #-}
```

```
import Data.Aeson
import Data.ByteString.Lazy.Char8 qualified as B
import GHC.Generics (Generic)

data Person = Person { name :: String, value :: Int }
  deriving stock (Show, Eq, Ord, Generic)
  deriving anyclass (ToJSON, FromJSON)
         -- anyclass derive


-- e.g.
encode $ Person "a" 1
          -- { name: "a", value: 1 }
decode @Person "{\"name\":\"a\", \"value\": 1 }"
           -- Person "a" 1
```

Method 2) default with **options**: i.e. simple **customization**, such as
e.g. capitalize all keys

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE DeriveAnyClass #-}

import Data.Aeson
import GHC.Generics (Generic)
import Data.Char (toUpper)

data Person = Person { name :: String, value :: Int }
  deriving stock (Show, Eq, Ord, Generic)

options :: Options
options = defaultOptions { fieldLabelModifier = (toUpper
<$>) }

instance ToJSON Person where
  toJSON = genericToJSON options
instance FromJSON Person where
  parseJSON = genericParseJSON options

-- e.g.
encode $ Person "a"
1                                              -- { NAME:
"a", VALUE: 1 }
decode @Person "{\"NAME\":\"a\", \"VALUE\":
1 }"                  -- Person "a" 1
```

Method 3) **manual**: i.e. implement **class toJSON** and **class fromJSON**
classes manually
Need to implement

```
 -- decoding
```

class FromJSON MyData where
**fromJSON** :: Value -> Parser a REQUIRED
Decodes by
ByteString --[Parser
Value] --> Value --[Parser a (fromJSON)]--> a

-- encoding
class ToJSON MyData where
**toJSON** :: a -> Value REQUIRED
Encodes by
a --[toJSON] -->
Value --[Encoding]--> ByteString
Inefficient since it needs
to use an intermediate Value type.
Required for backwards
compatibility

**toEncoding** :: a -> Encoding RECOMMENDED
More efficient by
encoding directly
a --[Encoding
(toEncoding)]--> ByteString

e.g.

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE RecordWildCards #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE DeriveAnyClass #-}

import Data.Aeson
import Data.Map qualified as M

data Person = Person { name :: String, value :: Int }
  deriving stock (Show, Eq, Ord)

instance FromJSON Person where
  parseJSON = withObject "Person" $ \obj -> Person <$>
obj .: "name" <*> obj .: "value"

instance ToJSON Person where
  toJSON Person {..} = Object $ "name" .= name <>
"value" .= value
  toEncoding Person {..} = pairs $ "name" .= name <>
"value" .= value

-- e.g.
encode $ Person "a"
1                                          -- { name:
"a", value: 1 }
decode @Person "{\"name\":\"a\", \"value\":
1 }"                  -- Person "a" 1
```

JSONKey

```
-- Data.Aeson
-- Encode
class ToJSONKey a where
 toJSONKey :: ToJSONKeyFunction a
 default :: ToJSON a => ToJSONKeyFunction

 toJSONKeyList :: ToJSONKeyFunction [a]
 default :: ToJSON a => ToJSONKeyFunction [a]

data ToJSONKeyFunction a =
  ToJSONKeyText :: !(a -> Text) !(a -> Encoding)
  ToJSONKeyValue :: !(a -> Value) !(a -> Encoding)


-- Decode
class FromJSONKey a where
 fromJSONKey :: FromJSONKeyFunction a
 fromJSONKeyList :: FromJSONKeyFunction [a]
```

**JSONKey**
Used to convert **JSON <--> Map** (Data.Map https://hackage.haskell.org/package/containers/docs/Data-Map.html)
In general, represents the key of a **JSON object**.
e.g. { a: 1 }  key="a"
But unlike Data.Map keys, JSON object keys only allow **strings**.
If the key cannot be converted to a string, uses **JSON array** instead.
e.g.

```
import Data.Aeson
import Data.Map qualified as M

encode $ M.singleton (1 :: Int) True
  -- { "1": true }
encode $ M.singleton ("1" :: String) True
 -- { "1": true }
encode $ M.singleton (1 :: Int) True
  -- { "1": true }
encode $ M.singleton [ 1 :: Int, 2 ] True
 -- [[[1,2], true]]
encode $ M.singleton (1 :: Int,2 :: Int)
True           -- [[[1,2], true]]
encode $ M.singleton (Just (1 :: Int)) True
-- ERROR. No instance of ToJSONKey (Maybe Int).
```

Basics

```
---- Value: i.e. represents JSON value
--
Null
String ""
Number 1
```

```
Bool True

-- Array
import Data.Vector qualified as V
{-# LANGUAGE OverloadedList #-}
Array :: Vector -> Value
Array $ V.fromList [Number 1, Null, Bool True]
  -- JSON [1, null, true]
Array [Number 1, Null, Bool True]
  -- same with OverloadedList

-- Object
import Data.Aeson.KeyMap qualified as K
{-# LANGUAGE OverloadedStrings #-}
Object $ K.fromList [("a", Number 1), ("b", Bool
True)]          -- JSON {a: 1, b: true}
Object $ "a" .= 1 <> "b" .= True
          -- same with class KeyValue
object :: [Pair] -> Value
object ["a" .= 1, "b" .= True]
          -- same
```

==========
Usages

Example: Default encode/decode data from JSON
Can just derive with **anyclass** deriving.

```
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}

import Data.Aeson
import GHC.Generics

data MyData1 where
  MyData1 ::
    { someInt :: Int,
      someString :: String
    } ->
    MyData1
  deriving stock (Show, Eq, Generic)
  deriving anyclass (FromJSON, ToJSON)
```

e.g.

```
encode $ MyData 1 "ABC"
             -- "{ \"someInt\": 1, \"someString\":
```

```
\"ABC\" }"
decode' @MyData "{ \"someInt\": 1, \"someString\":
\"ABC\" }"      -- Just (MyData 1 "ABC")
decodeEither' @MyData "{}"
          -- Left "Error in $: parsing
Main.MyData1(MyData1) failed, key \"someInt\" not found"
```

Example: specify encode/decode options

Use **Options**

e.g. convert from underscore

{ some_field :: 1 }  <->  MyData { someField = 1 }

```
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE OverloadedStrings #-}

import Data.Aeson
import GHC.Generics

data MyData where
  MyData :: {someField :: Int} -> MyData
  deriving stock (Show, Eq, Generic)

instance FromJSON MyData where
  parseJSON = genericParseJSON $ defaultOptions
    { fieldLabelModifier = camelTo2 '_'
  -- some_field -> someField
    }

instance ToJSON MyData where
  toEncoding = genericToEncoding $ defaultOptions
    { fieldLabelModifier = camelTo2 '_'
  -- someField -> some_field
    }
```

e.g.

```
encode $ MyData { someField = 1 }                  --
 "{ \"some_field\": 1 }"
decode' @MyData "{ \"some_field\": 1 }"            --
Just (MyData 1)
```

Usage: **manually** implement encode/decode

Manually convert to/from **Value** type

e.g.

data Person = Person { name :: String, value :: Int }

```
    p :: Person
    Object $ "name" .= name p <> "value" .= value p  :: Value
encode Person -> Value

    o :: Object
    Person <$> obj .: "name" <*> obj .: "value" :: Parser Person
 decode Value -> Person
```

```
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE RecordWildCards #-}

import Data.Aeson
import Data.ByteString.Lazy.Char8 qualified as B

data Person = Person { name :: String, value :: Int }
  deriving (Show, Eq)

-- Decode
instance FromJSON Person where
  parseJSON = withObject "Person" $ \obj -> Person <$>
obj .: "name" <*> obj .: "value"        -- Approach 1
(recommended): use with* function

  parseJSON (Object obj) = Person <$> obj .: "name" <*>
obj .: "value"                          -- Approach 2:
manual. Need to handle all cases of Value
  parseJSON _ = fail "Invalid"


-- Encode
instance ToJSON Person where
  toJSON Person {..} = Object $ "name" .= name <>
"value" .= value
  toEncoding Person {..} = pairs ["name" .= name,
"value" .= value]               -- Not required but
recommended for efficiency


-- e.g. use
main :: IO ()
main = do
  B.putStrLn $ encode $ Person "a" 1
  print $ decode @Person $ "{\"name\":\"b\", \"value\":
10 }"
  pure ()
```

e.g. output

```
{"name":"a","value":1}
Just (Person {name = "b", value = 10})
```

Usage: parse **literal** string with **Unicode**

Need to encode the literal in **UTF8** with Data.Text.Encoding

```
ghci> import Data.ByteString.Char8 qualified as B
ghci> import Data.Text.Encoding as T
ghci> import Data.Aeson

-- ERROR
ghci> eitherDecodeStrict @Value $ B.pack "{\"a\": \"アイ
ウエオ\" }"
Left "Error in $: Failed reading: Cannot decode input:
Data.Text.Internal.Encoding.decodeUtf8: Invalid UTF-8
stream. Expecting object value at '}'"

-- OK
ghci> eitherDecodeStrict @Value $ T.encodeUtf8 "{\"a\":
\"アイウエオ\" }"
Right (Object (fromList [("a",String
"\12450\12452\12454\12456\12458")]))
```

GOTCHA: parsing **(a OR b)** with **<|>** only reports error of last
Check each alternative  manually wit **case** instead of **<|>**

e.g. { "a1": true } OR { "a2": "hello" }

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE OverloadedStrings #-}

import Control.Applicative
import Data.Aeson
import Data.Aeson.KeyMap
import Data.Aeson.Types
import Data.Text (Text)
import Prelude hiding (lookup)

data MyDataBad1 = C1 Bool | C2 Text deriving (Eq, Show)

data MyDataBad2 = B1 Bool | B2 Text deriving (Eq, Show)

data MyDataGood = A1 Bool | A2 Text deriving (Eq, Show)

instance FromJSON MyDataBad1 where
  parseJSON = withObject "MyDataBad1" $ \obj -> do
    C1 <$> obj .: "c1" <|> C2 <$> obj .: "c2"
             -- BAD. Only reports errors for c2

instance FromJSON MyDataBad2 where
  parseJSON = withObject "MyDataBad2" $ \obj -> do
    B1 <$> obj .: "b1" <|> B2 <$> obj .: "b2" <|> fail
"b1 | b2"         -- BAD. All errors become fail "b1 |
b2"
```

```
 -  instance FromJSON MyDataGood where
  parseJSON = withObject "A" $ \obj -> do
    case lookup "a1" obj of
      Just v -> withBool "a1" (return . A1) v <?> Key
"a1"
      Nothing -> case lookup "a2" obj of
        Just v -> withText "a2" (return . A2) v <?> Key
"a2"
        Nothing -> fail "a1 or a2"
```

e.g.

```
-- BAD: only shows error of last option
print $ eitherDecode @MyDataBad1 "{\"c1\": true }"
print $ eitherDecode @MyDataBad1 "{\"d\": 1 }"     --
Left "Error in $: key \"c2\" not found"
print $ eitherDecode @MyDataBad1 "{\"c1\": 1 }"    --
Left "Error in $: key \"c2\" not found"
print $ eitherDecode @MyDataBad1 "{\"c2\": 1 }"    --
Left "Error in $.c2: parsing Text failed, expected
String, but encountered Number"

-- BAD: drops error of each option
print $ eitherDecode @MyDataBad2 "{\"b1\": true }"
print $ eitherDecode @MyDataBad2 "{\"d\": 1 }"      --
Left "Error in $: b1 OR b2"
print $ eitherDecode @MyDataBad2 "{\"b1\": 1 }"     --
Left "Error in $: b1 OR b2"
print $ eitherDecode @MyDataBad2 "{\"b2\": 1 }"     --
Left "Error in $: b1 OR b2"

-- GOOD
print $ eitherDecode @MyDataGood "{\"a1\": true }"
print $ eitherDecode @MyDataGood "{\"d\": 1 }"      --
Left "Error in $: a1 or a2"
print $ eitherDecode @MyDataGood "{\"a1\": 1 }"     --
Left "Error in $.a1: parsing a1 failed, expected
Boolean, but encountered Number"
print $ eitherDecode @MyDataGood "{\"a2\": 1 }"     --
Left "Error in $.a2: parsing a2 failed, expected String,
but encountered Number"
```

GOTCHA: parsing a **nested object** (e.g. {a: {b: {c: ... } } }) does not provide correct **path** in error message "Error in $ ..."
**JSONPath** (i.e. path of nested objects) is NOT preserved by bind (>>=)
Hence, need to wrap the whole inner parser in **<?>**

```
   p, q :: Parser a
   p <?> Key "a" >> q <?> Key "b"            BAD. Path "a" is not
preserved in q.
   (p >> q <?> Key "b") <?> Key "a"          GOOD. Path "a.b"
```

e.g.

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE OverloadedStrings #-}

import Data.Aeson
import Data.Aeson.Types

newtype MyBadData = MyBadData Bool deriving stock (Show,
Eq)
newtype MyGoodData = MyGoodData Bool deriving stock
(Show, Eq)

instance FromJSON MyBadData where
  parseJSON = withObject "base" $ \obj -> do
    (Object a) <- obj .: "a"
    (Object b) <- a .: "b"                    -- BAD.
"a" is not preserved
    (Bool c) <- b .: "c"                      -- BAD .
"b" is not preserved in error message
    return $ MyBadData c

instance FromJSON MyGoodData where
  parseJSON = withObject "base" $ \obj -> do
    (Object a) <- obj .: "a"
    do
      (Object b) <- a .: "b"
      do
        (Bool c) <- b .: "c"
        return $ MyData c
        <?> Key "b"                           -- GOOD
      <?> Key "a"                             -- GOOD.
Wrap whole
```

```
main :: IO ()
main = do
  -- GOOD
  print $ eitherDecode @MyGoodData "{}"
-- Left "Error in $: key \"a\" not found"
  print $ eitherDecode @MyGoodData "{ \"a\": {} }"
-- Left "Error in $.a: key \"b\" not found"
  print $ eitherDecode @MyGoodData "{ \"a\": { \"b\":
{} } }"         -- Left "Error in $.a.b: key \"c\" not
found"
  print $ eitherDecode @MyGoodData "{ \"a\": { \"b\":
{ \"c\": true } } }"

  -- BAD
  print $ eitherDecode @MyBadData "{}"
-- Left "
  print $ eitherDecode @MyBadData "{ \"a\": {} }"
-- Left "Error in $: key \"b\" not found"
  print $ eitherDecode @MyBadData "{ \"a\": { \"b\":
```

```
{} } }"          -- Left "Error in $: key \"c\" not
found"
  print $ eitherDecode @MyBadData "{ \"a\": { \"b\":
{ \"c\": true } } }"
```